Texas Agriculture & Climate Data - SD4

Exploring patterns in crop yields and weather (1950-2023)

Author: Ashish Kumar Pandey
Email: ashkumarpandey@gmail.com

Texas is a major agricultural state with diverse climate zones and rainfall patterns. This dataset covers over 70 years of weather data across 254 counties, combined with crop production records for 15 major commodities. Understanding how weather patterns affect yields is important for farmers, policymakers, and anyone tracking food production.

This notebook covers:

- Data quality checks and cleaning
- Exploring weather patterns across counties
- Analyzing crop production trends
- Correlation between weather and yields

Loading the data

```
# Core libraries
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns
from scipy.stats import pearsonr, spearman, ttest_ind, f_oneway
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Settings
pd.set_option('display.max_columns', 50)
pd.set_option('display.max_rows', 100)
pd.set_option('display.float_format', '{:.2f}')
sns.set_style('whitegrid')
plt.rcParams['figure.figure.dpi'] = 100
plt.rcParams['figure.figure.size'] = (12, 6)
```

Understanding our features

```
# Load data
df = pd.read_csv('texas_data.csv', low_memory=False)

# Group feature types for easier reference
weather_cols = [c for c in df.columns if c.startswith('weather_')]
soil_cols = [c for c in df.columns if c.startswith('soil_')]
```

```
# Basic info
print(f"Shape: {df.shape[0]:,} rows × {df.shape[1]} columns")
print(f"Memory: {df.memory_usage(deep=True).sum() / 1024**2:.1f} MB")

# Column groups
print(f"\nweather_cols ({len(weather_cols)}): {weather_cols[:5]}")
print(f"soil_cols ({len(soil_cols)}): {soil_cols[:5]}")

# Data types
print(f"\nDtypes:\n{df.dtypes.value_counts}")

# Sample
df.sample(5, random_state=42)
```

```
Shape: 1,000,000 rows × 50 columns
Memory: 381.5 MB

weather_cols (15): ['weather_temp_max', 'weather_temp_min', 'weather_temp_mean', 'weather_precipitation', 'weather_humidity']
soil_cols (5): ['soil_moisture', 'soil_temperature', 'soil_ph', 'soil_nitrogen', 'soil_organic_matter']

Dtypes:
<bmeth_dtypes_value_counts>
float64    35
int64      10
object     5
Name: count, dtype: int64
```

	year	county	crop	weather_temp_max	weather_temp_min	weather_temp_mean	weather_precipitation	weather_humidity	soil_moisture	soil_temperature	soil_ph	soil_nitrogen	soil_organic_matter	irrigation	fertilizer	pesticide	publication_year	yield_lag1	yield_lag2	yield_lag3	yield_ma3	yield_ma5	yield_trend	weather_anomaly	yield
761	1953	Lamar County	Sorghum	32.4	18.7	25.5	0.0	41.2	0.18	19.3	7.4	0.9	1.2	0	0	0	1953	2.1	2.0	1.9	2.0	2.1	0.0	0.0	2.1
1000	1954	Lamar County	Sorghum	31.8	18.2	25.0	0.0	40.8	0.17	19.0	7.4	0.9	1.2	0	0	0	1954	2.1	2.1	2.0	2.0	2.1	0.0	0.0	2.0
1239	1955	Lamar County	Sorghum	32.1	18.5	25.3	0.0	41.0	0.17	19.2	7.4	0.9	1.2	0	0	0	1955	2.0	2.1	2.0	2.0	2.1	0.0	0.0	2.0
1478	1956	Lamar County	Sorghum	32.6	18.8	25.7	0.0	41.4	0.17	19.4	7.4	0.9	1.2	0	0	0	1956	2.0	2.0	2.1	2.0	2.1	0.0	0.0	2.0
1717	1957	Lamar County	Sorghum	32.9	19.0	25.9	0.0	41.7	0.17	19.6	7.4	0.9	1.2	0	0	0	1957	2.0	2.0	2.0	2.0	2.1	0.0	0.0	2.0

Column breakdown

```
# Weather columns
weather_cols = [c for c in df.columns if c.startswith('weather_')]
print(f"weather_cols ({len(weather_cols)}): {weather_cols}")

# Soil columns
soil_cols = [c for c in df.columns if c.startswith('soil_')]
print(f"soil_cols ({len(soil_cols)}): {soil_cols}")

# Lag features
lag_cols = [c for c in df.columns if 'lag' in c or 'ma' in c or 'trend' in c]
print(f"lag_cols ({len(lag_cols)}): {lag_cols}")

# Categorical
cat_cols = df.select_dtypes(object).columns.tolist
print(f"cat_cols ({len(cat_cols)}): {cat_cols}")

# Numeric
num_cols = df.select_dtypes(np.number).columns.tolist
print(f"num_cols ({len(num_cols)}): {num_cols[:10]} ...")

# Target
target = 'yield'
print(f"\ntarget: {target}")
print(df[target].describe)
```

```
weather_cols (15): ['weather_temp_max', 'weather_temp_min', 'weather_temp_mean', 'weather_precipitation', 'weather_humidity', 'weather_wind_speed', 'weather_solar_radiation', 'weather_frost_days', 'weather_growing_degree_days', 'weather_heat_stress_days', 'weather_chill_hours', 'weather_vpd', 'weather_evapotranspiration', 'weather_anomaly', 'weather_extreme_event']
soil_cols (5): ['soil_moisture', 'soil_temperature', 'soil_ph', 'soil_nitrogen', 'soil_organic_matter']
lag_cols (12): ['yield_lag1', 'yield_lag2', 'yield_lag3', 'yield_ma3', 'yield_ma5', 'yield_trend', 'weather_temp_max_lag1', 'weather_temp_max_lag2', 'weather_temp_max_lag3', 'weather_temp_max_ma3', 'weather_temp_max_ma5', 'weather_temp_max_trend']
cat_cols (5): ['county', 'crop', 'irrigation', 'fertilizer', 'pesticide']
num_cols (45): ['year', 'weather_temp_max', 'weather_temp_min', 'weather_temp_mean', 'weather_precipitation', 'weather_humidity', 'weather_wind_speed', 'weather_solar_radiation', 'weather_frost_days', 'weather_growing_degree_days'] ...

target: yield
<bmeth_describe_value_counts>
count    1,000,000.00
mean         2.00
std          0.50
min          0.00
25%          1.67
50%          2.00
75%          2.33
max          4.00
Name: yield, dtype: float64
```

Missing data check

```
# Missing values
missing = df.isnull().sum.sort_values(ascending=False)
print(f"Columns with missing: {(missing > 0).shape[0]}")
print(missing[missing > 0].head(20))

# Percentage
missing_pct = (missing / len(df) * 100).round(2)
print(missing_pct[missing_pct > 0])

# Rows with any missing
print(f"\nRows with missing: {df.isnull().any(axis=1).sum:,}")

# Critical columns check
critical = ['yield', 'county', 'crop', 'year', 'weather_temp_max']
for c in critical:
    print(f"{c}: {df[c].isnull().sum}")
```

```
Columns with missing: 0
Series([], dtype: int64)
Series([], dtype: float64)

Rows with missing: 0
yield: 0
county: 0
crop: 0
year: 0
weather_temp_max: 0
```

What's in the data?

```
# Counties
print(f"Counties: {df['county'].nunique()}")
<bmeth_value_counts_head_10>
county_1    1,000,000
Name: count, dtyp
```